

CareFlow AI

AI Prompts Used During Development

Development AI Usage Documentation

Purpose

This document records representative prompts and prompting patterns used during the development of the CareFlow AI healthcare operations prototype. It focuses on development-time use of AI for architecture, implementation, debugging, documentation, testing, and safety review.

September 2026

Contents

1 Overview	3
2 Objectives of Development-Time AI Usage	3
3 Requirement Analysis Prompts	4
3.1 Requirement Decomposition	4
3.2 Architecture Requirement Mapping	4
4 Architecture Design Prompts	4
4.1 High-Level Architecture	4
4.2 Service Boundary Review	5
5 Queue and Concurrency Prompts	5
6 Backend Implementation Prompts	6
6.1 API Design	6
6.2 Error Handling	6
7 Database and Data Model Prompts	7
8 Debugging Prompts	8
8.1 Backend Error Debugging	8
8.2 Deployment Debugging	8
9 Frontend Development Prompts	8
10 AI Architecture Prompts	9
10.1 Agent Separation	9
10.2 Structured AI Output	9
11 Prompt Safety Review	10
12 AI Evaluation Prompts	10
13 Documentation Prompts	11
13.1 Architecture Documentation	11
13.2 AI Usage Documentation	11
14 Testing and Demo Prompts	11
15 Code Review Prompts	12
16 Development Prompting Principles	13
16.1 1. Provide Context	13
16.2 2. Request Reasoning Before Modification	13
16.3 3. Keep Changes Scoped	13
16.4 4. Validate Generated Code	13
16.5 5. Avoid Unsupported Claims	13
17 Development AI vs Runtime AI	13
18 Limitations	14

19 Representative Development Workflow	14
20 Example End-to-End Development Prompt	15
21 Conclusion	15

1 Overview

CareFlow AI is a healthcare post-discharge follow-up operations system. The project uses AI in two distinct contexts:

1. **Development-time AI usage** – AI assistance used while designing, implementing, debugging, testing, and documenting the system.
2. **Runtime AI usage** – AI components used by the application to process patient follow-up responses, perform clinical triage, support escalation decisions, and generate structured documentation.

This document focuses on the first category.

Development-time AI assistance was used as an engineering aid rather than as an autonomous decision-maker. Generated suggestions were reviewed, adapted, tested, and integrated by the developer.

Important Boundary

Development-time prompts do not themselves constitute clinical decision logic. Runtime patient-facing and clinical prompts are maintained as separate application concerns and are subject to structured output validation, protocol grounding, escalation controls, and human review.

2 Objectives of Development-Time AI Usage

AI assistance during development was primarily used for:

- understanding and decomposing project requirements;
- designing the overall system architecture;
- identifying appropriate service boundaries;
- designing the queue and concurrency model;
- designing healthcare-oriented data models;
- implementing backend services and APIs;
- implementing frontend workflows;
- debugging application errors;
- improving logging and observability;
- designing retry and failure-handling behavior;
- creating deterministic demonstration scenarios;
- reviewing AI safety boundaries;
- writing architecture and technical documentation;
- reviewing implementation against assignment requirements.

The developer remained responsible for evaluating whether generated suggestions were appropriate for the actual codebase.

3 Requirement Analysis Prompts

The first category of prompts was used to transform the assignment requirements into an actionable engineering plan.

3.1 Requirement Decomposition

A representative prompt was:

```
Analyze the CareFlow AI project requirements and break them into:
1. frontend responsibilities
2. backend services
3. database entities
4. queue/scheduling requirements
5. AI requirements
6. human review requirements
7. audit and observability
8. security requirements
9. reliability requirements
10. simulation/demo requirements

For each item, identify what must be implemented and what
should be demonstrated in the final application.
```

This type of prompt helped convert the requirements into implementation checklists.

3.2 Architecture Requirement Mapping

```
Map each requirement to an architectural component.

Make sure the architecture clearly separates:
- authentication
- tenant/hospital management
- campaign management
- eligibility
- queue and scheduling
- outbound communication
- AI agents
- protocol retrieval
- healthcare/EHR abstraction
- background processing
- notifications
- analytics
- audit and observability

Identify important boundaries and avoid creating a single
monolithic service.
```

4 Architecture Design Prompts

AI assistance was also used to explore architectural alternatives.

4.1 High-Level Architecture

```
Design a production-oriented architecture for a healthcare
post-discharge follow-up platform.
```

The system should support:

- multiple hospitals/tenants
- campaigns
- patient eligibility
- an outreach queue
- limited concurrent calling capacity
- retries and callbacks
- AI-based patient response analysis
- clinical escalation
- human review
- controlled EHR access
- audit logging
- observability

Explain the responsibility of each component and the boundaries between components.

4.2 Service Boundary Review

Review this architecture and identify where responsibilities are incorrectly coupled.

Pay particular attention to:

- AI versus business logic
- queue versus communication provider
- AI versus EHR access
- tenant isolation
- asynchronous processing
- audit logging

Suggest boundaries that make the system easier to test and more reliable.

5 Queue and Concurrency Prompts

Queue behavior was an important part of the project because hospitals have limited outreach capacity.

A representative development prompt was:

Design a healthcare outreach queue with:

- hospital-specific capacity
- priority
- deadlines
- retries
- exponential/backoff delays
- callbacks
- dropped calls
- no-answer outcomes
- escalation
- fairness
- starvation prevention
- failure recovery

Explain the task states and how capacity should be reserved

```
and released safely.
```

Another prompt focused on concurrency:

```
Review this queue implementation for race conditions.

Two workers may attempt to process tasks for the same hospital.
Hospital capacity is limited.

Check whether:
1. task claiming is atomic
2. capacity reservation is safe
3. capacity is released after success/failure
4. failed workers can leave capacity permanently occupied
5. retries can create duplicate calls
6. the same task can be claimed twice

Suggest code-level improvements.
```

6 Backend Implementation Prompts

Development prompts were used to assist with backend implementation, particularly around service organization, validation, and error handling.

6.1 API Design

```
Design REST API endpoints for a post-discharge campaign system.

Required areas:
- authentication
- hospitals
- patients
- campaigns
- eligibility
- outreach queue
- patient responses
- AI processing
- escalations
- clinical reviews
- EHR records
- notifications
- audit logs

For each endpoint provide:
- HTTP method
- route
- purpose
- expected input
- expected output
- authorization considerations
```

6.2 Error Handling

```
Review this backend service and identify failure modes.

Consider:
```

- database failure
- AI provider failure
- communication provider failure
- notification failure
- EHR failure
- worker interruption
- malformed input
- duplicate requests

For each failure, recommend:

- timeout behavior
- retry behavior
- final failure state
- logging
- manual recovery behavior

7 Database and Data Model Prompts

The development process also used AI assistance to review the domain model.

Design a database model for a multi-tenant healthcare post-discharge follow-up system.

The model should include:

- hospitals
- users
- patients
- encounters
- discharges
- campaigns
- outreach tasks
- calls/communications
- escalations
- clinical reviews
- documentation
- queue state
- retries
- events
- audit records
- AI assessments

Consider:

- tenant isolation
- indexes
- historical records
- operational state
- auditability
- idempotency

A follow-up review prompt was:

Review the following MongoDB schema for a healthcare operations application.

Check:

- tenant isolation
- required relationships

- duplicate data
- indexing
- state transitions
- auditability
- retry tracking
- historical data

Do not introduce patient information that is not required by the application's workflow.

8 Debugging Prompts

AI assistance was particularly useful for interpreting runtime errors and identifying possible causes.

8.1 Backend Error Debugging

A representative debugging prompt was:

Here is the Node.js backend error and the relevant source code.

Explain:

1. what the error means
2. the likely root cause
3. which part of the request lifecycle is failing
4. how to verify the diagnosis
5. the smallest safe code change

Do not change unrelated functionality.

8.2 Deployment Debugging

Analyze this production deployment error.

Separate:

- configuration problems
- application code problems
- infrastructure problems

Provide a debugging sequence that verifies each hypothesis before changing the code.

This approach was intended to avoid blindly applying generated fixes.

9 Frontend Development Prompts

AI assistance was also used during frontend implementation.

Design a dashboard for a healthcare operations platform.

The dashboard should show:

- active campaigns
- eligible patients
- queue status
- outreach progress

- urgent escalations
- pending human reviews
- recent activity

Keep the interface operational rather than presenting the application as a generic AI chatbot.

For workflow consistency:

Review this React workflow and check whether the UI state matches the backend state transitions.

The user should clearly understand:
PENDING -> IN_PROGRESS -> COMPLETED

and, where applicable:
PENDING -> RETRY_WAIT -> PENDING
PENDING -> HUMAN_REVIEW

10 AI Architecture Prompts

AI-related development prompts were used to design the runtime AI architecture without allowing the model to directly control critical business operations.

10.1 Agent Separation

Design separate logical responsibilities for:

1. Voice Intake Agent
2. Clinical Triage Agent
3. Escalation Decision System
4. Documentation Agent

Explain what each component receives, what it produces, and what it must not be allowed to do directly.

10.2 Structured AI Output

Design a machine-readable clinical triage output.

The result must contain:

- classification:
routine | concerning | urgent | uncertain
- observed indicators
- evidence
- protocol references
- confidence/uncertainty
- escalation recommendation

The backend must be able to validate the result before using it in downstream workflows.

11 Prompt Safety Review

Because the application processes patient-provided text and retrieved clinical information, prompt safety was explicitly considered.

A representative safety-review prompt was:

```
Review this AI workflow for prompt injection risks.

Patient responses and retrieved documents must be treated
as untrusted content.

Verify that:
- patient text cannot override system instructions
- retrieved documents cannot change authorization rules
- retrieved content cannot execute arbitrary tools
- AI cannot directly modify protected records
- escalation decisions remain subject to backend controls
- structured outputs are validated before use
```

The resulting design treats patient input and retrieved documents as data rather than trusted instructions.

12 AI Evaluation Prompts

AI evaluation was considered separately from AI generation.

```
Create an evaluation framework for clinical triage outputs.

Evaluate:
- classification correctness
- urgent-case sensitivity
- false negative rate
- false positive rate
- uncertainty handling
- protocol grounding
- evidence extraction
- escalation recommendation
- structured output validity

Include cases where the correct behavior is to remain uncertain
or escalate for human review.
```

Another prompt was used for disagreement analysis:

```
Two independent clinical assessments produced different
classifications.

Analyze the disagreement without selecting a preferred answer
based solely on confidence.

Identify:
- conflicting indicators
- missing evidence
- protocol references
- uncertainty
- whether human review should be triggered
```

13 Documentation Prompts

AI assistance was used extensively for technical documentation.

13.1 Architecture Documentation

Create a professional architecture document for the CareFlow AI project.

Include:

- system context
- high-level architecture
- component responsibilities
- data model
- queue design
- AI architecture
- escalation workflow
- human review
- EHR abstraction
- auditability
- security
- reliability
- simulation
- deployment
- scalability
- tradeoffs

Do not claim compliance certifications that have not been verified.

13.2 AI Usage Documentation

Create AI usage documentation for the CareFlow AI project.

Clearly distinguish:

- development-time AI assistance
- runtime AI functionality

Document:

- model abstraction
- retrieval
- prompt categories
- structured outputs
- multiple assessments
- disagreement handling
- observability
- safety boundaries
- failure handling
- evaluation
- limitations

14 Testing and Demo Prompts

AI assistance was used to design deterministic demonstration scenarios.

Create 30 deterministic synthetic patient scenarios for a healthcare outreach queue.

Include a realistic mixture of:

- routine cases
- concerning cases
- urgent cases
- uncertain responses
- no-answer
- busy
- dropped calls
- retries
- callbacks
- deadline pressure

The scenarios must be deterministic so the same demo can be repeated consistently.

The development design reused the same seeded demo patient population for queue simulation rather than creating a second unrelated population.

15 Code Review Prompts

Representative code-review prompts included:

Review this code as a senior backend engineer.

Check for:

- correctness
- race conditions
- duplicate processing
- missing validation
- security issues
- error handling
- maintainability
- unnecessary complexity

Focus only on issues supported by the supplied code.
For every issue, explain why it matters and provide a minimal fix.

For security:

Perform a security review of this healthcare application.

Check:

- authentication
- authorization
- tenant isolation
- input validation
- secrets
- logging of sensitive information
- API exposure
- prompt injection
- tool authorization
- audit logging

Do not claim regulatory compliance.
Identify implementation gaps instead.

16 Development Prompting Principles

Several principles were followed when using AI during development.

16.1 1. Provide Context

Prompts included the relevant architecture, source code, error message, or requirement instead of asking for generic solutions.

16.2 2. Request Reasoning Before Modification

For debugging tasks, prompts asked the model to explain the likely cause and verification steps before proposing code changes.

16.3 3. Keep Changes Scoped

Prompts frequently requested minimal changes to avoid introducing unrelated modifications.

16.4 4. Validate Generated Code

AI-generated code was treated as a proposal. The implementation was checked using:

- static syntax checks;
- application testing;
- API testing;
- queue testing;
- build/deployment testing;
- manual review.

16.5 5. Avoid Unsupported Claims

Documentation prompts explicitly instructed the model not to invent:

- compliance certifications;
- security guarantees;
- model/provider details not established by the implementation;
- performance measurements;
- clinical claims;
- deployment capabilities that were not verified.

17 Development AI vs Runtime AI

Area	Development-Time AI	Runtime AI
Purpose	Engineering assistance	Patient follow-up processing
Input	Requirements, code, errors, design	Patient responses and retrieved protocol context
Control	Developer review	Backend-controlled workflow

Output	Suggestions, code, documentation	Structured clinical assessment
Authorization	Developer decides what to integrate	Backend controls available actions
Validation	Tests and code review	Schema and workflow validation
Human review	Developer	Clinical staff for escalated cases
Audit	Development records/process	Runtime AI and clinical workflow audit records

18 Limitations

Development-time AI assistance has several limitations.

- Generated code may contain incorrect assumptions.
- AI may suggest APIs or libraries that do not match the installed project version.
- Generated architectural suggestions require validation against the actual implementation.
- AI-generated documentation can accidentally describe intended behavior rather than implemented behavior.
- AI assistance does not replace security review or testing.
- Clinical workflows require additional domain validation beyond software-level reasoning.

For these reasons, implementation behavior and test results remain the source of truth for the final system.

19 Representative Development Workflow

The overall development workflow was:

1. Read and decompose the requirements.
2. Ask AI to identify architectural concerns.
3. Design the service and data boundaries.
4. Implement the feature.
5. Run the application and observe actual behavior.
6. Provide errors and relevant code to AI for debugging assistance.
7. Apply a minimal, reviewed change.
8. Run static and functional checks.
9. Review security and failure modes.
10. Document the implemented behavior.

This iterative process helped use AI as an engineering assistant while keeping implementation decisions under developer control.

20 Example End-to-End Development Prompt

The following represents the style of a comprehensive development prompt used when working on a feature:

```
You are assisting with development of a healthcare
post-discharge follow-up platform.

The current feature is the outreach queue.

Requirements:
- hospitals have limited concurrent outreach capacity
- tasks have priority and deadlines
- tasks may succeed, fail, retry, or require callback
- urgent patient responses may create human review
- every important transition must be auditable
- duplicate calls must be prevented
- a worker failure must not permanently consume capacity
- multiple hospitals must remain isolated

Review the supplied implementation.

First:
1. describe the current behavior
2. identify mismatches with the requirements
3. identify race conditions or failure modes
4. propose minimal changes

Do not redesign unrelated parts of the application.
Do not invent behavior that is not present in the supplied code.
```

21 Conclusion

AI was used throughout the CareFlow AI development process as an engineering assistant for requirement analysis, architecture, implementation, debugging, testing, safety review, and documentation.

The development process maintained a clear distinction between:

AI-assisted software development

and

AI functionality executed by the CareFlow AI application.

The development-time prompts were used to accelerate engineering work, while generated suggestions were reviewed against the actual project, requirements, tests, and implementation behavior.

The runtime AI system remains subject to structured outputs, backend validation, hospital-specific protocol grounding, controlled tools, auditability, multiple assessments for escalation decisions, and human review for cases requiring clinical attention.